Experiment 10

Student Name: Suyash
UID: 25MCI10054
Branch: MCA (AI/ML)
Section/Group: 25MAM-1(A)
Semester: 2-SEM
Date of Performance: 21/04/2026
Subject Name: Technical Training - 1
Subject Code: 25CAP-652

1. Aim of the Session:

To understand and apply transaction control in PostgreSQL using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT to maintain data integrity during database operations.

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

Apply transaction control commands for safe and reliable database updates.

4. Theory:

Transaction control in PostgreSQL ensures that a group of database operations is executed safely as a single unit. Using **BEGIN**, a transaction starts; **COMMIT** permanently saves the changes, while **ROLLBACK** undoes them if an error occurs. **SAVEPOINT** allows partial rollback to a specific point within a transaction. These commands help maintain data integrity and consistency, especially during complex or multi-step operations.

5. Practical / Experiment Steps:

Step 0: Table creation and data insertion.

Code:

```
Create table payroll(
emp_id int primary key,
emp_name varchar(50),
```

```
salary decimal(10,2) check(salary>0)
);

Insert into payroll values
(1,'Amit',30000),
(2,'Neha',40000),
(3,'Ravi',50000);
```

Output:

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	1	Amit	30000.00
2	2	Neha	40000.00
3	3	Ravi	50000.00

Step 1: Transaction with savepoint.

Code:

```
begin;

update payroll
set salary=salary+5000
where emp_id=1;

savepoint sp1;

update payroll
set salary=salary+7000
where emp_id=2;

update payroll
set salary=-1000
where emp_id=3;
```

rollback to sp1;

commit;
select * from payroll;

Output:

```
-- Update 1
UPDATE Payroll
SET salary = salary + 5000
WHERE emp_id = 1;

-- Savepoint created after first valid update
SAVEPOINT sp1;

-- Update 2
UPDATE Payroll
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 43 msec.
```

First valid update before savepoint

```
WHERE emp_id = 2;
```

Data Output | Messages | Notifications

```
SAVEPOINT

Query returned successfully in 47 msec.
```

Savepoint created successfully

```sql
-- Roll back only till savepoint
ROLLBACK TO sp1;

-- Commit the valid change
COMMIT;

-- Final table
SELECT * FROM Payroll;
```

Data Output Messages Notifications

	emp_id [PK] integer	emp_name character varying (50)	salary numeric (10,2)
1	2	Neha	40000.00
2	3	Ravi	50000.00
3	1	Amit	35000.00

Final result after rollback to savepoint and commit

6. Learning Outcomes (What I have learned):

- **BEGIN** starts a transaction block for grouping multiple SQL commands.
- **ROLLBACK** cancels changes made inside the transaction.
- **SAVEPOINT** allows partial rollback without losing all valid updates.
- **CHECK** constraint ensures salary remains greater than zero.
- **PostgreSQL** transaction control helps maintain consistency and data integrity.